

Technical Design Assignment

Client-Side **Tic Tac Toe** (React & TypeScript)

Background & Requirements

This project focuses on building a client-side Tic Tac Toe application using React and TypeScript. While the problem itself is simple, the main challenge lies in modeling the game rules correctly and keeping the logic predictable and testable.

The game supports:

- A 3x3 game board
- Standard Tic Tac Toe (fixed marks like 'X' and 'O' per player)
- Wild Tic Tac Toe (players choose mark on each turn)
- Local play for two human players
- Human vs. computer mode

While the computer player is not required to be perfect, it must at least:

- Play a winning move when available
- Block obvious opponent wins

Given the **~10-hour** implementation constraint, the focus is intentionally on:

- Ensuring the correctness of the game rules
- Keeping the game logic easy to test
- Accessibility for all users
- Clear and maintainable structure

These aspects are more important than setting up complex infrastructure or non-essential features.

Architecture Overview

Due to the client-side nature of the application, the game is built as a **SPA** to ensure immediate UI responsiveness and maintain game state without page reloads.

The application follows a lightweight separation, clean-architecture-inspired structure. The goal here is not to fully implement clean architecture, but to separate responsibilities enough so that the core game logic remains framework-independent and easy to test.

The system is split into four layers:

- **UI Layer** — React components and presentation
- **Application Layer** — state management and orchestration
- **Domain Layer** — game rules and core logic (heart of the application)
- **Infrastructure Layer** — handles external concerns, especially data persistence (session storage)

Dependencies flow strictly in one direction:

UI → Application → Domain
└─ Infrastructure Adapter

What matters most is that:

- The **Domain layer** does not depend on React or browser APIs
- The **Application layer** coordinates behavior but does not implement rules
- UI components remain simple and focused on interaction

This structure ensures that the code is highly testable, has clear separation of concerns, and can be easily expanded in the future (e.g., support online multiplayer).

As illustrated in **Figure 1**, the **Domain layer** remains isolated from both **UI** and **Infrastructure** concerns, while the **Application layer** acts as the **central orchestration point**.

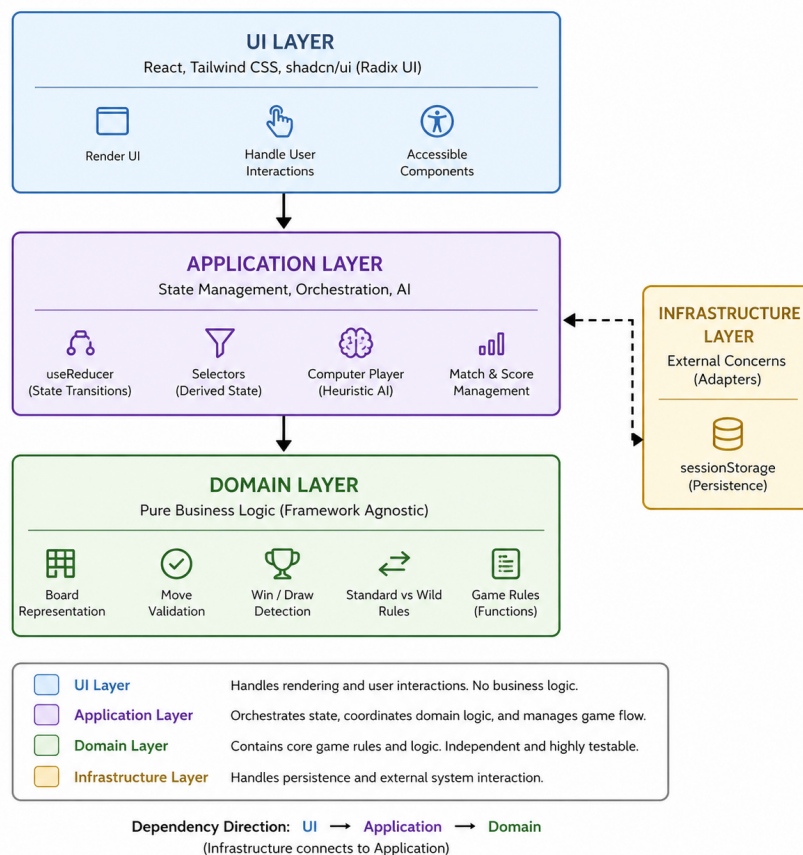


Figure 1: High-level architecture showing the separation between UI, application logic, domain rules, and infrastructure. The domain layer remains isolated, while infrastructure interacts only with the application layer.

Technology Stack

Core

- **React 19 (latest)**
Used for building the UI and managing component rendering. React's concurrent features are not required here due to the small scope, but the latest version ensures modern APIs and long-term compatibility.
- **TypeScript**
Provides strong typing for the domain model and reduces the risk of invalid state transitions.

UI Layer

- **Shadcn/ui on top of Radix UI**
Used for building accessible UI components quickly.
Benefits:
 - Accessible by default (keyboard + screen reader support)
 - Avoids reinventing interaction patterns
 - Unstyled primitives allow full control over UI
 - Reduces time spent on accessibility edge cases
- **Tailwind CSS**
Used for styling and responsive layout.

Code Quality

- **Biome (linter + formatter)**

Chosen over ESLint because:

- Fewer dependencies and less configuration boilerplate
- Faster setup and execution
- Unified tool (linting + formatting)

For a small project, minimizing tooling overhead is preferred.

Problem Statement 1: Modeling Game Rules

The challenge is to support both **Standard** and **Wild** Tic Tac Toe without duplicating logic or introducing UI complexity.

Considered Approaches

Approach A — Logic inside React components

Pros:

- Fastest way to implement

Cons:

- Tightly couples UI code with the game's business logic
- Difficult to test game logic in isolation
- Error-prone for Wild mode

Approach B — Pure domain model (chosen)

Game rules are implemented as pure TypeScript functions operating on a small, explicit game model.

At a high level, the Domain model includes:

- **Board:** the current state of the grid, representing all played and available cells
- **Move:** the selected cell, chosen mark, and player {`cellIndex`, `mark`, `playerId`}
- **Game status:** in-progress, win, or draw
- **Game mode:** Standard or Wild

The key design decision is that a move always includes both the selected cell and the mark being placed. This makes Wild mode a natural extension of the same model, rather than a special case handled inside the UI.

The **Domain layer** is responsible for:

- Validating moves
- Applying moves
- Detecting wins and draws
- Enforcing Standard vs. Wild rules
- Preventing moves after the game has ended

This keeps the rule logic centralized and allows both human and computer moves to be validated through the same path.

Decision

The **Domain-Driven approach** is chosen because it:

- Centralizes all rule logic
- Keeps UI simple
- Enables thorough unit testing
- Avoids duplication between human and computer logic

This ensures that all game invariants are enforced in a single place.

Problem Statement 2: Computer Player Design

The goal is to implement a computer player that is good enough, without over-investing in complexity.

Considered Algorithms

1. Random Move (Rejected)

The computer selects any available move randomly.

Pros:

- Trivial implementation

Cons:

- Fails to take winning moves
- Does not block opponent
- Produces weak gameplay

Conclusion: Too simplistic, does not meet requirements.

2. Depth-Limited Minimax (Strong Alternative)

Evaluates future game states to choose the best move.

Pros:

- Strong gameplay
- Handles multi-turn strategy

Cons:

- Higher implementation complexity
 - Harder to test thoroughly
 - Wild mode significantly increases branching factor
 - Overkill for stated requirements
-

3. Rule-Based Heuristic AI (Chosen)

The Heuristic approach reuses the same Domain rule engine (e.g., winner detection and move validation), ensuring consistent behavior between human and computer moves.

Uses prioritized decision rules:

1. Play winning move
2. Block opponent winning move
3. Take center
4. Take corner
5. Take any available cell

Wild mode adaptation:

- Each move is evaluated as {cellIndex, mark, playerId}
- Both X and O are tested per cell

Pros:

- Satisfies requirements
- Simple and explainable
- Easy to test
- Integrates directly with Domain logic

Cons:

- Not perfect
 - May miss multi-step strategies
-

Additional Considerations (Not Implemented)

- Monte Carlo simulation (overkill)
 - Pattern-based strategies (less flexible in Wild mode)
-

Final Decision

The **heuristic approach (No3)** is selected because it:

- Meets all functional requirements
- Is easy to reason about
- Minimizes implementation risk
- Fits the time constraint

Minimax was considered the **closest alternative** but was rejected due to complexity vs. benefit trade-off.

Additional Technical Challenge: State Management

The application must manage game state predictably without introducing unnecessary complexity.

Considered Approaches

Approach A — Global state (Zustand/Redux)

Pros:

- Scalable

Cons:

- Unnecessary for small app
- Adds setup overhead

Approach B — React `useReducer` (chosen)

Pros:

- Built-in
- Explicit state transitions
- Easy to test

Cons:

- Less scalable long-term

Decision

The application uses React's **`useReducer`** to:

- Manage game state transitions
- Use it as a single source of truth
- Coordinate domain logic
- Keep dependencies minimal

The reducer also acts as a **guard** against invalid state transitions (e.g., preventing moves after game completion), implicitly enforces turn sequencing (human → computer), preventing duplicate or out-of-order actions, and ensuring that all state changes follow a predictable and valid flow.

Selectors are used to derive UI-friendly state and belong to the **application layer**.

Derived values such as winner or available moves are not stored but computed from the current state, reducing the risk of inconsistencies.

Normalized vs Denormalized State

Given the very small and fixed dataset, normalization would add unnecessary complexity and abstraction without improving performance or flexibility.

Accessibility

Accessibility is treated as a **core requirement**, not an enhancement.

Key design decisions:

- Board implemented using semantic `<button>` elements and is treated as grid of interactive elements rather than a visual layout
- Full keyboard navigation supported
- Visible focus states
- Disabled cells cannot be re-selected
- Game status is announced via **aria-live**
- Mark selection is implemented using accessible, keyboard-navigable controls based on Radix UI primitives (via shadcn/ui), which follow **WAI-ARIA** guidelines.

Additional considerations:

- No reliance on color alone
- Clear textual feedback for all states
- Consistent interaction patterns

Accessibility is prioritized early to avoid retrofitting later.

Testing Strategy

Testing focuses on correctness and user interaction.

Unit Tests (Highest Priority)

Since all game correctness is centralized in the domain layer, the majority of confidence in the system comes from thorough unit testing of these core rules.

Cover Domain logic:

- Win detection (all directions)
- Draw detection
- Move validation
- Standard vs. Wild behavior
- Computer decision logic (win/block)

Integration Tests

Using **React Testing Library**:

- Starting a game
 - Making moves
 - Switching modes
 - Human vs. computer flow
 - Score updates
-

Accessibility Testing

Verify:

- Keyboard navigation
 - Accessible labels
 - Screen reader announcements
 - Correct disabled states
-

E2E Testing (Playwright)

E2E tests are considered but not implemented due to time constraints.

Planned coverage:

- Full game flow
- Human vs. computer gameplay

Reason for exclusion:

- Lower priority compared to Domain correctness
 - Higher setup cost relative to time budget
-

Final Conclusion

Must have

- Strong Domain unit tests
- “Computer” win/block tests
- **React Testing Library** happy paths
- Accessibility checks for keyboard and aria-live

Nice-to-have

- Playwright smoke tests

Optional: Persistence Strategy

Persistence is treated as a nice-to-have, **not** a core requirement.

If time allows, **sessionStorage** will be used for non-critical state such as selected game mode, player mode, match score, and games played.

The active game board remains in memory to keep the core game flow simple and avoid persistence-related edge cases. If implementation time becomes tight, persistence will be removed before reducing coverage of game rules, “Computer” behavior, or accessibility.

Other concerns

Responsiveness

The application is designed to be **fully responsive** and usable across mobile, tablet, and desktop devices. Tailwind CSS is used to ensure layout flexibility.

Routing & URL State

No significant data is stored in the URL, since there are no deep-linking requirements. In this context, storing state in the URL would add unnecessary complexity.

This could be reconsidered though, if sharing game state becomes a requirement.

Performance Considerations

The maximum number of possible game states is small, which guarantees consistent performance regardless of interaction patterns or how frequently the computer evaluates possible moves. All operations are $O(1)$ or $O(n)$ with $n=9$, making performance effectively constant.

Security Considerations

Even though this is a client-only application, basic security awareness is maintained. React's default escaping prevents most XSS risks unless unsafe HTML is explicitly introduced. Since all inputs are constrained to predefined game interactions, the potential attack surface remains minimal. Issues like CSRF and Clickjacking are not applicable for this context due to the absence of server interaction.

Technology Decisions (What Was Not Used and Why)

These tools were considered, but intentionally **excluded** to keep the proof of concept aligned with the time constraint and required feature set.

✗ Nx / Monorepo / pnpm Workspaces

Not used because:

- Single application scope
- No shared packages
- Added setup complexity
- No immediate benefit

Would be appropriate if:

- Multiple apps existed
 - Shared Domain packages were required
-

✗ Firebase / Supabase

Not used because:

- Remote multiplayer is **not required**

- Introduces networking complexity (create rooms, 2 players must join, etc.)
- Requires handling:
 - a. Synchronization
 - b. Presence
 - c. Reconnection
 - d. Conflict resolution

Would be appropriate if a **real-time multiplayer** was required.

Risk and Mitigation

The following table summarizes the main risks identified during the design and the corresponding mitigation strategies.

Risk	Impact	Mitigation
Overengineering	Time spent on non-essential work	Keep architecture lightweight
Wild mode complexity	Incorrect gameplay	Explicit move model + tests
“Computer” complexity creep	Less time for testing	Use heuristic approach
Accessibility gaps	Poor usability	Use semantic + Radix
Persistence inconsistencies	Confusing session behavior	Scoped sessionStorage

A.I Usage

I used ChatGPT as a writing and reasoning assistant during this submission. It helped me to improve the clarity, structure, and overall readability of this document. Additionally, the LLM assisted in organizing the content to better reflect my intended prioritization and logical flow.

Example prompts:

```
- Review whether the proposed architecture is appropriate for a small almost 10 hours react app implementation.
- Review this technical design as if you were a senior frontend engineer and identify weak areas and improve structure, readability and fix grammar-typos.
- Give more emphasis 1 accessibility (important for modern apps) and 2 testing section. About testing, unit testing is most important and E2E less important must be excluded due to time limitations.
```

All final technical decisions, assumptions, prioritization, and trade-offs were reviewed and selected by me.